

```
>arrayArg=00000000000100
Entered [2 ] with st->val=33 , st-
>arrayArg=000000000000033
Entered [3 ] with st->val=16 , st-
>arrayArg=000000000000016
Entered [4 ] with st->val=5 , st-
>arrayArg=000000000000005
Entered [5 ] with st->val=1 , st-
>arrayArg=000000000000001
Entered [6 ] with st->val=0 , st-
>arrayArg=000000000000000
[raman@Chalotra Part2]$
```

Note that the output may vary across executions, because of the random number used in reducing the recursive problem.

Auxiliary program

Here is *auxSymbols.c*:

```
#include<stdio.h>
/* The UDT 'auxST' is same as UDT 'ST' used by traceMe_rel
 *
 */
typedef struct auxType{
    int auxVal;
    int auxArray[15];
} auxST;

/* Following function is just to make sure
 * that there is a user of 'auxST',
 * so that GCC does not optimise 'auxST'
 *
 * This function would never be called!
 */
int fakeFunc(auxST *ptr )
{
    auxST *ptr_1;
    ptr->auxVal++;
    printf ("\"n %s\", ptr->auxVal);
}
```

Compile *auxSymbols.c* with `gcc -g auxSymbols.c -c` and `gcc -g-shared -PIC auxSymbols.c -o libauxSym.so`. (The first creates an object file *auxSymbols.o*, the second a shared library *libauxSym.so*.) Note that both commands pass `-g` to GCC, to build in debug mode, which is very important. Note down the size of *auxSymbols.o* with a `ls -al auxSymbols.o`; we will need this later on.

'Function parameter logging' scripts

Let's apply the technique discussed in the *Basic Concepts*

section as we will write three scripts:

- `'trace1.gdb'`: This script uses *auxSymbols.o* to log the function parameters.
- `'trace2.gdb'`: Uses *libauxSym.so* to log function parameters.
- `'trace2_mod.gdb'`: Modified *trace2* to support function argument modification.

I have declared an auxiliary UDT with the name *auxST* that is equivalent to *ST* (used in *traceMe_rel*, for the function `'ST *'`). So, here is what we do:

1. Load *traceMe_rel* in GDB. At this point, type information for *ST* is not known.
2. Load symbols from auxiliary object code (*auxSymbols.o*) or the shared library (*libauxSym.so*), compiled in debug mode.
3. Put a breakpoint on function *func*, and extract the argument using the processor register *EBP*, which acts as the base pointer.
4. Once the argument is extracted, cast it to the *auxST* type and extract the sub-types, i.e., *auxVal* and *auxArray* (the *val* and *arrayArg* fields of the UDT *ST*).
5. Play with the arguments as you wish: print them, test them against some condition, modify them, etc. You can even stop the program from executing.

Now, let's dig into the interesting details, one by one.

'trace1.gdb'

```
file ./traceMe_rel
br main
run 100
bt
call open("auxSymbols.o", 2)
set variable $fd=$
call mmap(0, 2516, 1|2|4, 1, $fd,0)
set variable $_address=$

print "\n $fd=%d, $_address=0x%x\n", $fd, $_address
add-symbol-file auxSymbols.o $_address
ptype auxST
ptype func
br func
c
while 1
set variable $_one= *((int *)($ebp +0x8))

print "\n GDB:TRACER: %d, %s\n", ((auxST *) ($_one))->auxVal,
((auxST *) ($_one))->auxArray
c
end
```

This script uses the `add-symbol-file` command of GDB. The *auxSymbols.o* file is opened using the *open* system call